

Lang Graph → library - langchain

↳ Graph Based / Nodes Based

↓
Easy - Simple

↓
Stateful → Maintain State

langchain - framework

↳ composable
→ Modular

[Agents → Agents]

Complex

↳ S1 → S2 → S3

↳ 4 → 0/0

Agent workflow

Query

↓
Supervisor

Telecom Agent

Ecommerce Agent

Educat. agent

Composed Agent

→ O/P Agent

Complex

↳ Retries

→ Conditional Branching

→ State Management

Components

① Node

→ function / Executable

② Edge

→ Path → workflow direction

③ Conditional Edge

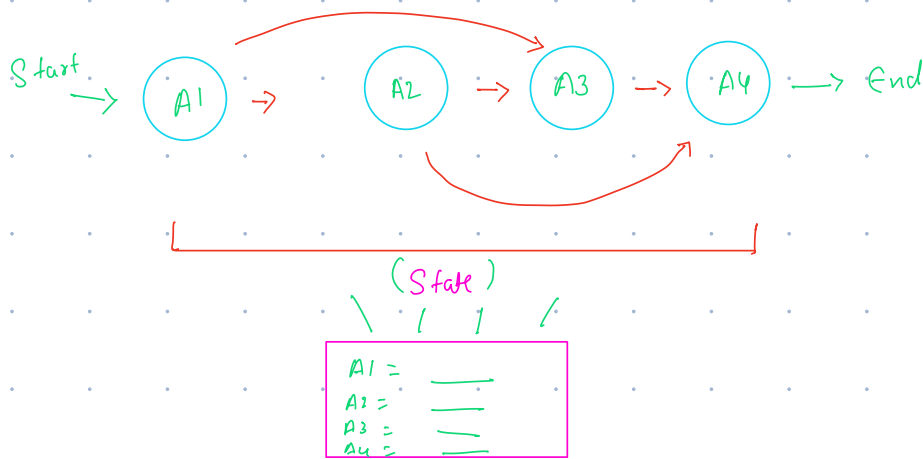
→ Condition Based Routing

N1 → N2
↓
N3 → N4

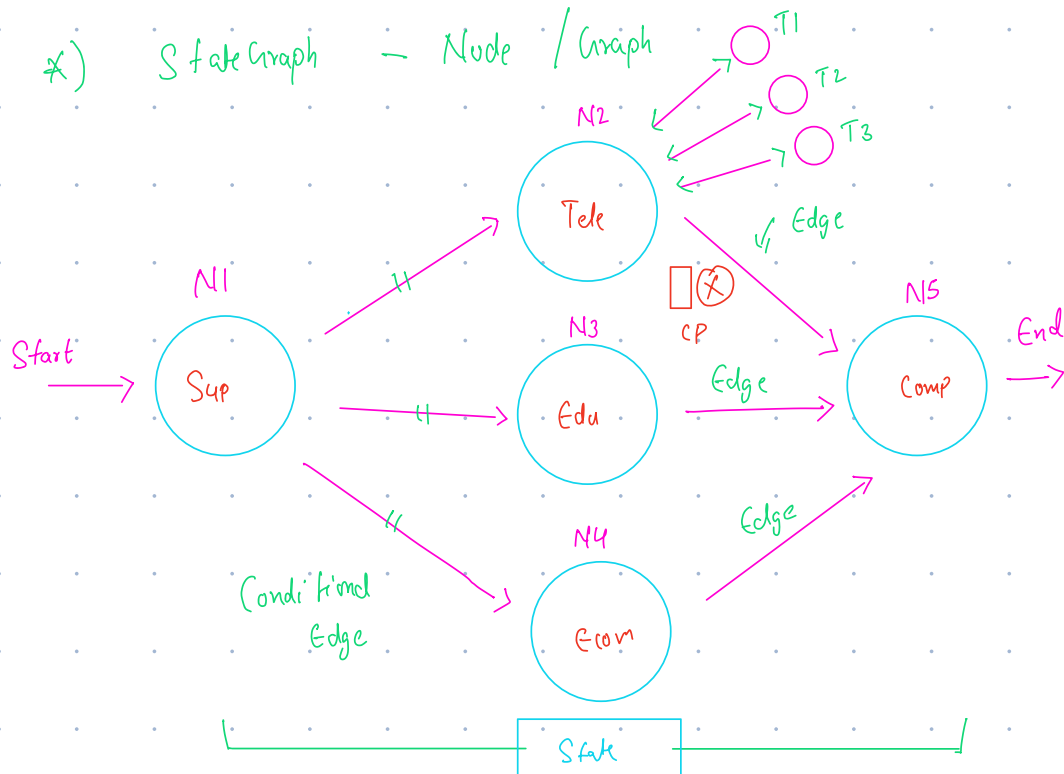
N1 → N3
N1 → N4

④ State / Agent state → Stateful workflow

⑤ Start / End → Entry to workflow / End



* State Graph - Node / Graph



long term - ✓
short term - ✓
episodic term - ✓

S - _____
T - _____
e - _____
e - _____

Runtime - State

Checkpoints

CP - CP - CP - CP
 ✓ ✓ (X)
 ✓ ✓ - W/F

(State) Management → current World
 ↓
 APP.

State * Memory

• lifecycle state
 { what is weather
 in london

State :=
 message { role : user
 ↓
 weather : None
 answer : None
 }

State
 cannot writing data
 changes
 shared - All Nodes

Memory
 preserved
 run across
 session
 often long
 lived
 personalized
 persistence.

Tool Run

weather : {
 city : london
 temp : 31
 cond : sunny
 }

→ LLM →

Answer =

```
from typing import TypedDict
```

```
class AgentState(TypedDict):  
    messages: list  
    memory: dict  
    context: dict
```

Contract

```
class AgentState(TypedDict):
```

```
    messages: list  
    memory: dict  
    context: dict  
    retrieved_docs: list  
    search_results: list  
    current_plan: str  
    final_answer: str
```

Conversation - Context

```
memory = {  
    "user_name": "Ahmed",  
    "favorite_language": "Python",  
    "preferred_editor": "VS Code",  
    "experience": "Intermediate"  
}
```

```
messages = [  
    {  
        "role": "system",  
        "content": "You are a helpful assistant."  
    },  
    {  
        "role": "user",  
        "content": "Explain Python."  
    },  
    {  
        "role": "assistant",  
        "content": "Python is a programming language."  
    },  
    {  
        "role": "user",  
        "content": "Who created it?"  
    }  
]
```

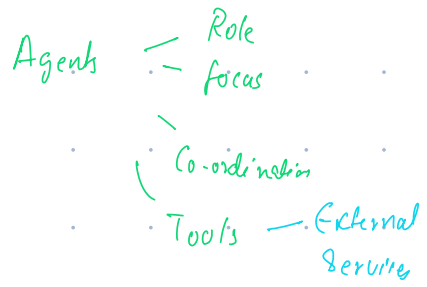
```
context = {  
    "current_document": "sales.pdf",  
    "page": 18,  
    "retry_count": 2,  
    "active_tool": "pdf_reader"  
}
```

```
class AgentState(TypedDict):  
    messages: list  
  
    # Long-term information  
    memory: dict  
  
    # Runtime metadata  
    context: dict  
  
    # Retrieved knowledge  
    retrieved_documents: list  
  
    # Tool outputs  
    search_results: list  
  
    # Planning  
    plan: str  
    completed_steps: list  
  
    # Current task status  
    active_tool: str  
    retry_count: int  
  
    # Final output  
    answer: str  
  
    # Error tracking  
    error: str | None
```

Tools

→ Agents

↳ Extend Capabilities



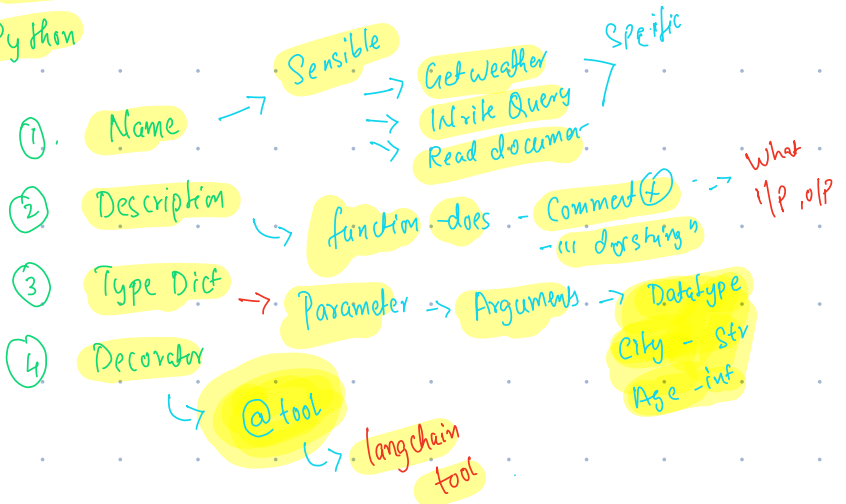
- ✓ (SQL Query)
 - ✓ Document
 - ✓ API call
 - ✓ Send Email
- LLM (Dumb)

Tools →

functions

Python

Binded → Agent
↓
Tool - Arsenal
Done



@tool

```
def get_weather (city: str):  
    """ gets you temperature by taking city as input """  
  
    _____  
    _____  
    _____  
  
    return
```

Tool function

- 1) Query
- 2) API
- 3) Intriking
- 4) Reading